

Dungeon Assistant

Ecosistema Inteligente de Gestión TTRPG

PWA Mobile-First con Inteligencia Artificial

Evaluación Parcial 3

Integrantes	Cristóbal Mira - Maverick Valdés - Francisco Toloza
Asignatura	TALLER APLICADO DE PROGRAMACIÓN
Docente	Felix Eduardo Cifuentes Cid
Fecha	Junio 2026

1. Introducción

Dungeon Assistant es una Progressive Web Application (PWA) diseñada para optimizar y enriquecer la experiencia de juego de Dungeons & Dragons 5ª Edición (D&D 5e). El sistema integra inteligencia artificial generativa, comunicación en tiempo real mediante WebSockets y reconocimiento óptico de caracteres (OCR) para automatizar las tareas más repetitivas del Game Master y los jugadores: toma de notas, gestión de inventario, seguimiento de personajes y consultas de reglas.

El proyecto es desarrollado por un equipo de tres estudiantes con roles diferenciados según sus especialidades técnicas: Cristóbal Mira en el rol de Full-Stack Lead & Product Owner, liderando la visión del producto y la arquitectura de IA; Maverick Valdés como Arquitecto Backend, a cargo del módulo OCR, WebSockets e interfaz responsiva; y Francisco Toloza como Full-Stack Developer, responsable de la enciclopedia de campaña y las funciones de WebSpeech.

El presente informe detalla el estado de avance actual del proyecto, cubriendo la metodología de desarrollo adoptada, los patrones de diseño implementados, la arquitectura del sistema, las tecnologías seleccionadas, la configuración de servidores, las integraciones con servicios externos y el diseño visual de la aplicación.

2. Índice

1. Introducción
2. Índice
3. Desarrollo
 - 3.1 Metodología de Desarrollo
 - 3.2 Patrones de Diseño
 - 3.3 Arquitectura del Sistema
 - 3.4 Lenguaje de Programación y Tecnologías
 - 3.5 Configuración de Servidor y Despliegue
 - 3.6 Aspectos de Integración
 - 3.7 Mockups y Experiencia Visual
 - 3.8 Anexos
4. Plan de pruebas
 - 4.1 Alcance de pruebas
 - 4.2 Pruebas fuera de alcance
 - 4.3 Estrategia de Pruebas
 - 4.4 Priorización de Componentes
 - 4.5 Resumen ejecutivo
- 5 Conclusion

3. Desarrollo

3.1 Metodología de Desarrollo

El proyecto se desarrolla bajo la metodología ágil Kanban, organizando el trabajo en un flujo continuo de tareas priorizadas por el valor de negocio. El equipo mantiene un tablero con columnas Por Hacer, En Progreso y Terminado, limitando el trabajo en curso para evitar cuellos de botella y permitir una respuesta flexible ante cambios durante el desarrollo. Esta metodología se escogió debido a que se adapta mejor a equipos pequeños al reducir la carga y mantener visibilidad constante del avance sin necesidad de ciclos fijos.

Para la recolección de información, se revisó la documentación oficial de las tecnologías base (FastAPI, React, Gemini API, dnd5eapi.co), el reglamento de D&D 5e como fuente de reglas de validación y plataformas referentes como D&D Beyond y Roll20.

Tablero Kanban y Políticas Explícitas

El sistema de flujo continuo se implementó mediante la definición de estados del tablero, el establecimiento de límites de Trabajo en Curso (WIP) para evitar cuellos de botella y la creación de políticas de transición entre columnas. Se definen cadencias de reposición de tareas (Replenishment) según la capacidad del equipo y reuniones de revisión de flujo para la mejora continua. El backlog funciona como un inventario de opciones disponible para ser jalado al flujo activo cuando el límite WIP lo permite.

Distribución de Responsabilidades

- **Cristóbal Mira** — Full-Stack Lead & Product Owner: lidera la visión del producto, la arquitectura de experiencia de usuario (UX/UI), la arquitectura del motor RAG, la integración de IA con Gemini y el modelo de base de datos.
- **Maverick Valdés** — Arquitecto Backend: responsable del módulo de OCR con Gemini Vision, la comunicación en tiempo real mediante WebSockets y la implementación de la interfaz web responsiva.
- **Francisco Toloza** — Full-Stack Developer: responsable del desarrollo de la enciclopedia de campaña e implementación de las funciones de WebSpeech.

Roadmap por Fases

- **Fase 1 — Definición del Proyecto:**
 - Levantamiento del problema, contexto, solución, arquitectura de software, stack tecnológico, modelo de base de datos y diseño del flujo de trabajo Kanban.
- **Fase 2 — Base, Autenticación y Dashboard:**
 - Autenticación (Login / Registro) con JWT ————— T-03, T-04, T-05.
 - Modelo relacional en PostgreSQL/Supabase con políticas RLS ————— T-02.

- Dashboard principal y hub de navegación ————— T-06.
- **Fase 3 — Campañas, Roles y NPCs:**
 - Creación de campañas con asignación automática de Game Master ————— T-07.
 - Sistema de roles por campaña con restricción de GM único ————— T-08.
 - CRUD completo de NPCs por campaña ————— T-09.
- **Fase 4 — RAG, IA, Personajes y Voz:**
 - Motor RAG con embeddings de campaña e indexación vectorial ————— T-10.
 - Integración de Gemini para respuestas narrativas contextualizadas ————— T-11.
 - Hoja de personaje D&D 5e completa ————— T-12.
 - CRUD de personajes por campaña ————— T-13.
 - Asistente de voz con WebSpeech API ————— T-14.
- **Fase 5 — OCR y PWA:**
 - Módulo OCR con Gemini Vision para digitalización de hojas físicas ————— T-15.
 - Configuración de Service Worker, manifest.json y responsividad completa — T-16.
- **Fase 6 — Despliegue:**
 - Deploy del frontend en Vercel ————— T-17.
 - Deploy del backend en Render con Docker y health checks ————— T-18.
- **Fase 7 — QA y Pruebas:**
 - Plan de pruebas con Postman y Pytest ————— T-19.
 - Ejecución de pruebas E2E y registro de resultados ————— T-20.

3.2 Patrones de Diseño

El sistema aplica patrones de diseño consolidados tanto en el backend asíncrono como en el frontend reactivo, garantizando escalabilidad, bajo acoplamiento y alta velocidad de respuesta.

Patrón	Ubicación / Clase	Propósito
Singleton	SupabaseClient(backend/services/supabase.py)	Garantiza una única instancia del cliente de base de datos en todo el ciclo de vida del servidor. Reutiliza conexiones tanto con credenciales ANON_KEY como SERVICE_KEY.
Dependency Injection	Depends() de FastAPI(routers y middleware)	Injecta recursos (cliente DB, usuario autenticado) en los enrutadores. Desacopla la infraestructura y facilita las pruebas unitarias.
Service / Manager Pattern	SocketManagerRAGManager	Extrae la lógica de negocio de los routers hacia clases de servicio autónomas, aislando el transporte HTTP/WebSocket de las reglas del sistema.
Middleware Pattern	backend/middleware/auth.py	Intercepta todas las peticiones HTTP, verifica el JWT en el encabezado Authorization y añade el contexto de usuario de forma transversal.
State Store (Zustand)	useAuthStoreuseCampaignStore	Implementa flujo unidireccional tipo Flux. Centraliza el estado global y sincroniza cambios reactivos entre componentes distantes sin prop drilling.
Optimistic UI	Registro de notas e inventario(frontend)	Añade elementos en la interfaz de forma instantánea antes de la confirmación del backend. Realiza rollback automático en Zustand si la petición falla.

3.3 Arquitectura del Sistema

DungeonAssistant adopta una Arquitectura Cliente-Servidor Desacoplada con persistencia en la nube, integraciones de IA híbridas y comunicación bidireccional en tiempo real mediante WebSockets.

Diagrama de Arquitectura General (véase en 3.8 Anexos)

Sistema de IA con RAG (Retrieval-Augmented Generation)

La inteligencia artificial de *DungeonAssistant* (Google Gemini) se apoya en una arquitectura RAG para solventar las tres limitaciones inherentes de los modelos de lenguaje: la falta de conocimiento sobre datos privados de la campaña, la alucinación de respuestas y el costo/límite de tokens en la ventana de contexto.

El RAG actúa como un puente de traducción cognitiva entre la base de datos relacional (Supabase) y el modelo generativo de la IA, operando bajo dos fases desacopladas:

A. Fase de Indexación y Población (Escritura en Background)

Cuando un jugador registra una nota de sesión, el backend no bloquea la experiencia de usuario. Guarda la nota instantáneamente e inicia una tarea asíncrona en segundo plano (*asyncio.create_task*):

- **Extracción Estructurada:** Llama a Gemini pasándole la nota con instrucciones de sistema estrictas para extraer de forma cognitiva NPCs y objetos, forzando la salida en un formato de datos JSON.
- **Población en Base de Datos:** El servidor procesa este JSON y actualiza la tabla de memoria a largo plazo *rag_entities*. Si el NPC u objeto ya existía, incrementa su contador de menciones (*mention_count = mention_count + 1*) y actualiza sus atributos; si es nuevo, crea el registro.

B. Fase de Recuperación y Consulta (Lectura en el Chat)

Cuando el usuario realiza una pregunta en el chat conversacional de la campaña (por ejemplo, *"/assistant/chat"*), **no se le inyecta la base de datos completa a la IA**, sino un fragmento ultra-condensado:

1. **Búsqueda Selectiva (Recuperación):** El servidor ejecuta consultas SQL en Supabase para obtener el Lore base (*campaigns*), las estadísticas de los personajes activos (*characters*) y las entidades RAG más populares de la campaña (*rag_entities* ordenadas descendientemente por menciones).

2. **Traducción y Compilación del Prompt:** El código del backend en Python traduce y formatea estas filas de base de datos relacionales en texto Markdown estructurado (inyectando variables de atributos, nombres y relaciones).
3. **Inyección y Generación:** Este bloque Markdown se inyecta en el prompt del sistema como "contexto inmutable". Gemini lee este bloque como si fuera un libro de lore y redacta una respuesta en español 100% coherente y libre de alucinaciones sobre el historial privado de la campaña.

Comunicación Bidireccional en Tiempo Real (Socket.io)

Para la gestión del tracker de combate en vivo y la sincronización de las salas de juego, el sistema no utiliza el protocolo HTTP tradicional (que requiere peticiones repetitivas del cliente), sino una conexión persistente:

- **Modelo Pub/Sub y Salas Virtuales:** Al iniciar sesión, el cliente WebSocket abre un canal directo con el servidor. El backend agrupa a los usuarios en salas virtuales dinámicas (*campaign_{campaign_id}*). Cuando el GM realiza una acción (por ejemplo, avanzar de turno o añadir un monstruo), el servidor emite el evento únicamente a los clientes pertenecientes a esa sala, reduciendo drásticamente el consumo de ancho de banda.
- **Mecanismo de Fallback:** Para garantizar el funcionamiento en redes móviles inestables, el protocolo se apoya en **Socket.io**. Si el WebSocket nativo TCP se bloquea por un cortafuegos, el sistema degrada la conexión automáticamente a *HTTP Long-Polling*, manteniendo la interactividad en tiempo real sin interrumpir la partida.

Rotación Dinámica de Modelos de Gemini

Dado que la API de Google Gemini impone límites estrictos de cuota por minuto (*Rate Limits*), la capa de servicios del backend implementa un patrón de **tolerancia a fallos y alta disponibilidad**:

- **Monitoreo y Conmutación por Error (Failover):** El servicio **GeminiService** envuelve cada llamada en un capturador de excepciones que analiza los códigos de error HTTP. Si la API de Google responde con un error de cuota agotada (**Código 429**) o modelo no disponible (**Código 404**), el backend activa un algoritmo de rotación en memoria, cambiando de forma transparente al siguiente modelo de la lista de respaldo (**gemini-3.5-flash** - **gemini-2.5-flash** - **gemini-1.5-flash**), y reintenta la petición en milisegundos sin que el usuario final perciba el fallo.

Seguridad: Row Level Security (RLS)

La seguridad de los datos no depende del backend exclusivamente. Supabase PostgreSQL evalúa políticas SQL directamente en la base de datos:

1. El usuario autenticado recibe un JWT firmado por Supabase Auth.

- 2. Al consultar tablas como *campaign_members*, *characters* o *session_notes*, la DB inyecta el ID del usuario (*auth.uid()*) en cada política.
- 3. Los roles son contextuales: se evalúa si un usuario es GM o PLAYER dentro de esa campaña específica, eliminando roles de administrador global inseguros.

3.4 Lenguaje de Programación y Tecnologías

El stack tecnológico fue seleccionado priorizando rendimiento asíncrono, compatibilidad con SDKs de IA y una experiencia de usuario mobile-first premium.

Capa	Tecnología	Detalle
Backend	Python 3.11+	Elegido por su soporte nativo para asyncio, FastAPI y SDKs de IA. Librerías: fastapi, pydantic-v2, python-socketio, supabase, uvicorn.
Frontend	JavaScript ES6+React 18	Componentes funcionales con Hooks modernos. Compilaciones inmediatas con Vite. Librerías: tailwind-css, zustand, socket.io-client, axios.
Base de Datos	PostgreSQL(Supabase)	Tipos JSONB avanzados para indexar análisis RAG rápidamente sin sacrificar estructura relacional. Auth y Storage integrados.
IA Generativa	Google Gemini1.5 Flash	Interacción contextualizada con el lore de la campaña, sugerencia de reglas y resúmenes inteligentes de sesión.
Tiempo Real	Socket.io(WebSockets)	Comunicación bidireccional autenticada. Salas por campaña, emisión de eventos de inventario, notas y roles.

3.5 Configuración de Servidor y Despliegue

DungeonAssistant adopta una estrategia de Despliegue Híbrido (Dual-Deployment) dentro de una estructura de Monorepo. Esto permite separar la entrega del cliente estático de la ejecución del servidor de tiempo real, optimizando costos y rendimiento en la nube.

Frontend— Vercel

El frontend desarrollado en React se compila estáticamente (produciendo assets HTML, CSS y JS optimizados) y se despliega en Vercel (<https://dungeon-assistant-test.vercel.app>)

Backend — Railway

Debido a que las arquitecturas Serverless tradicionales (como Vercel Functions o AWS Lambda) no soportan conexiones TCP persistentes y cortan los canales bidireccionales de larga duración, el servidor de producción principal se despliega en Railway.

- **Servicio Persistente:** Railway ejecuta un contenedor Dockerizado continuo definido mediante un archivo Procfile en el backend:

```
web: uvicorn main:app --host 0.0.0.0 --port 8000
```

- **Soporte nativo de WebSockets:** Al correr como un proceso continuo y no-serverless, el servidor Uvicorn mantiene abiertos los canales TCP de Socket.io indefinidamente, permitiendo una sincronización de combate de bajísima latencia para los jugadores.

3.6 Aspectos de Integración

DungeonAssistant orquesta múltiples APIs externas y servicios locales de forma coordinada:

- **Supabase Auth & Database (REST / JWT):** Control central de sesiones y almacenamiento persistente. Los tokens de seguridad del frontend se replican en el handshake del WebSocket para autenticar conexiones en tiempo real.
- **Google Gemini API (Generative & Vision SDK):** Integrado en el backend a través del SDK oficial de Google (*google-generativeai*). Esta API se explota en dos modalidades:
 - **Modelos Generativos (Gemini Flash):** Utilizado para ejecutar el motor de inferencia del RAG conversacional, generación dinámica de NPCs coherentes con el lore del juego y síntesis de resúmenes estructurados al cierre de sesión.
 - **Modelos de Visión (Gemini Vision):** Consume capacidades multimodales para recibir el buffer de bytes de imágenes de hojas físicas preprocesadas, ejecutando un OCR semántico que mapea los textos extraídos a esquemas estrictos de Pydantic.
- **dnd5eapi.co & Supabase Encyclopedia:** Conexión con la API REST de referencia pública oficial de D&D 5e. Para evitar latencias y permitir el soporte offline del juego, los datos estáticos de monstruos, conjuros y dotes se sincronizan y cachean localmente en la tabla *encyclopedia* de Supabase, utilizando la API externa como fallback secundario en caso de consultas no indexadas.
- **Web Speech API (Integración nativa del navegador):** Integración nativa del navegador del dispositivo móvil del jugador en el frontend de React. Al delegar el procesamiento de voz a texto directamente al chip del hardware local del usuario, se elimina la latencia de red, se evita el consumo de tokens de APIs de pago externas (como OpenAI Whisper) y se garantiza un funcionamiento fluido y de costo cero.
- **Socket.io (Protocolo WebSocket):** Canal en tiempo real autenticado para sincronizar salas de campaña (*campaign_{id}*). Emite eventos como `user_joined`, `session_started`, `note_added` y actualizaciones de inventario.

3.7 Mockups y Experiencia Visual

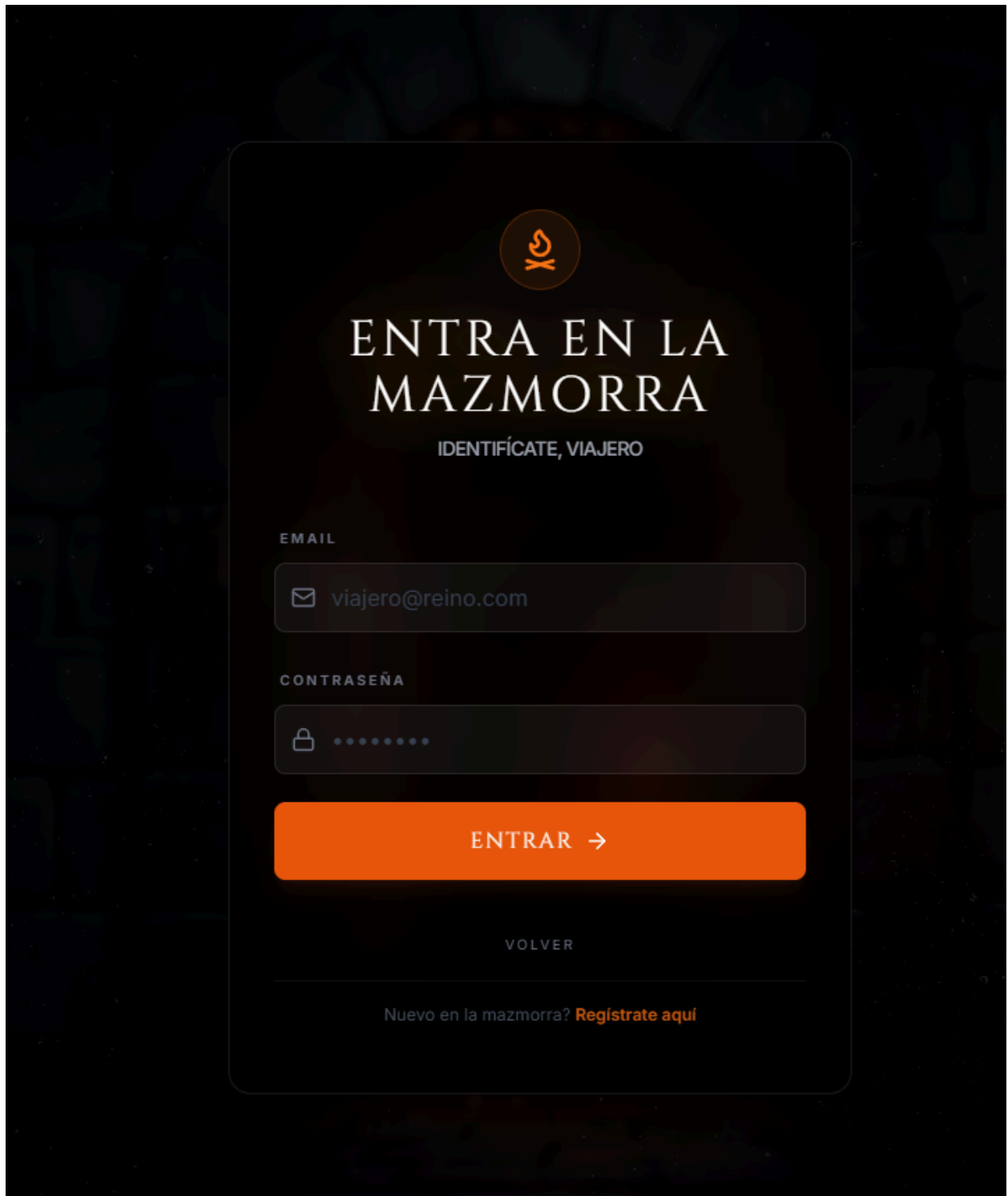
El diseño de DungeonAssistant apunta a una experiencia visual premium con temática medieval D&D, pensada desde el primer momento para dispositivos móviles (mobile-first). A continuación se describe cada vista principal de la aplicación. Las capturas de pantalla de la aplicación funcional se adjuntan como evidencia del estado de avance actual.

Pantalla de Bienvenida / Login

- Fondo inmersivo medieval oscuro /Login y Registro como pestañas dinámicas



Pantalla de inicio de sesión (Credenciales)



The login screen features a dark, textured background with a central white login form. At the top of the form is a circular logo containing a stylized orange flame and a cross. Below the logo, the title "ENTRA EN LA MAZMORRA" is displayed in a large, white, serif font, followed by the subtitle "IDENTIFÍCATE, VIAJERO" in a smaller, white, sans-serif font. The form includes two input fields: "EMAIL" with a placeholder "viajero@reino.com" and an envelope icon, and "CONTRASEÑA" with a placeholder of ten dots and a lock icon. A prominent orange button labeled "ENTRAR →" is positioned below the password field. At the bottom of the form, the word "VOLVER" is centered, and a link "Nuevo en la mazmorra? [Regístrate aquí](#)" is provided.



ENTRA EN LA MAZMORRA

IDENTIFÍCATE, VIAJERO

EMAIL

 viajero@reino.com

CONTRASEÑA



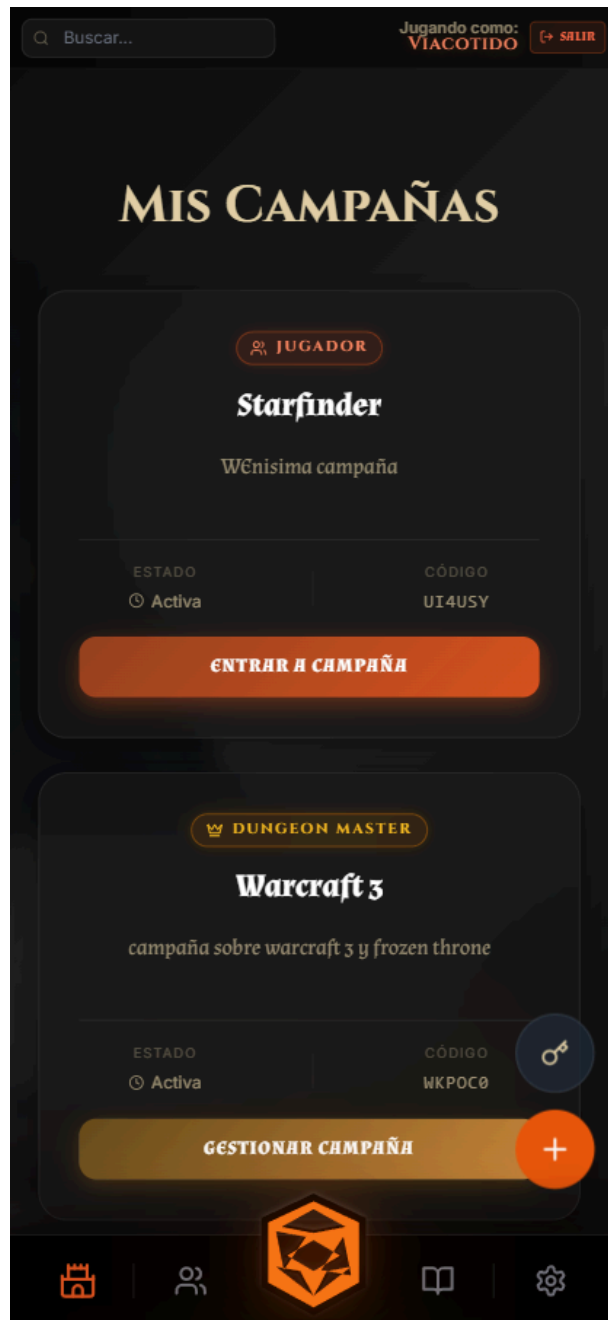
ENTRAR →

VOLVER

Nuevo en la mazmorra? [Regístrate aquí](#)

Dashboard del Jugador / GM

- Listado de campañas activas en tarjetas. (Mobile y Desktop)



MIS CAMPAÑAS

Bienvenido, frrrrr.

 Unirse con Código

 Nueva Campaña

 JUGADOR

Prueba Flujo

Una campaña sin descripción
aguarda por ti...

ESTADO

⌚ Activa

CÓDIGO

AE0L6U

ENTRAR A CAMPAÑA

 JUGADOR

Reino perdido

asd100

ESTADO

⌚ Activa

CÓDIGO

N9ZFPH

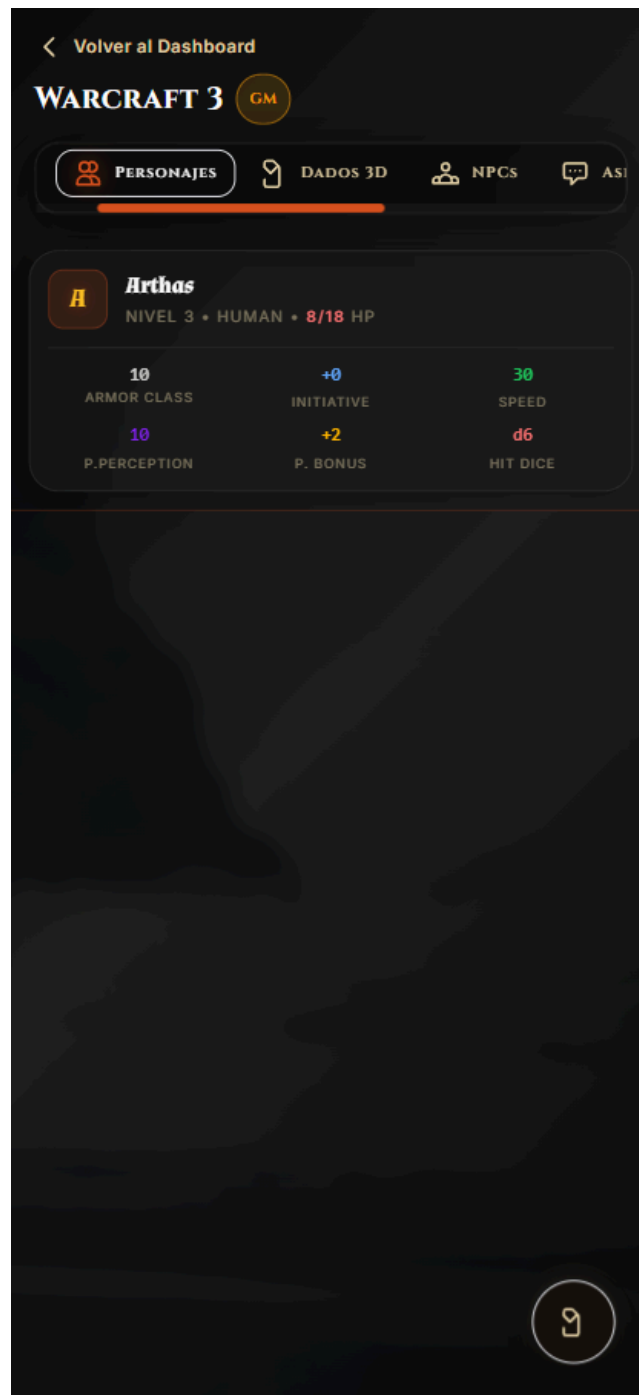
ENTRAR A CAMPAÑA



CREAR NUEVO MUNDO

Vista de Campaña Principal (CampaignView)

- **Mobile:** barra de navegación superior fija para alternar entre Notas de Sesión, Personajes, NPCs del Lore y Configuración.



- **Desktop:** menú lateral fijo izquierdo y panel extensible con funcionalidades de Notas, Personajes, Datos3D, NPCs, Asistente, Miembros, Configuración.

DUNGEON
ASSISTANT

CAMPAÑAS

PERSONAJES

ESTADÍSTICAS

ENCICLOPEDIA

RASGOS

EQUIPAMIENTO
Y OBJETOS

MONSTRUOS

HECHIZOS

DICE BOX

AJUSTES

Volver al Dashboard

WARCRAFT 3GM

NOTASPERSONAJESDADOS 3DNPCsASISTENTE MIEMBROS CONFIGURACIÓN

SESIONES

Sesión 1

Selecciona una sesión para ver sus notas

CERRAR SESIÓN

Sección Mis Personajes

Mis Personajes

Gestiona tus héroes, **Viacotido**. La gloria te espera.

+ AÑADIR A CAMPAÑA



NIVEL 10

Ross

HUMAN ROGUE

HP 53 / 53

10

ARMOR CLASS

+0

INITIATIVE

25

SPEED

10

PASSIVE PERCEPTION

+4

PROFICIENCY BONUS

1

HIT DICE (D6)



NIVEL 3

Hijo

HALFLING RANGER

HP 18 / 18

10

ARMOR CLASS

+0

INITIATIVE

30

SPEED

10

PASSIVE PERCEPTION

+2

PROFICIENCY BONUS

1

HIT DICE (D6)



NIVEL 15

Bubito

DRAGONBORN CLERIC

HP 78 / 78

10

ARMOR CLASS

+0

INITIATIVE

25

SPEED

10

PASSIVE PERCEPTION

+4

PROFICIENCY BONUS

1

HIT DICE (D6)



NIVEL 5

Nola

DWARF BARD

HP 28 / 28

10

ARMOR CLASS

+0

INITIATIVE

30

SPEED

10

PASSIVE PERCEPTION

+2

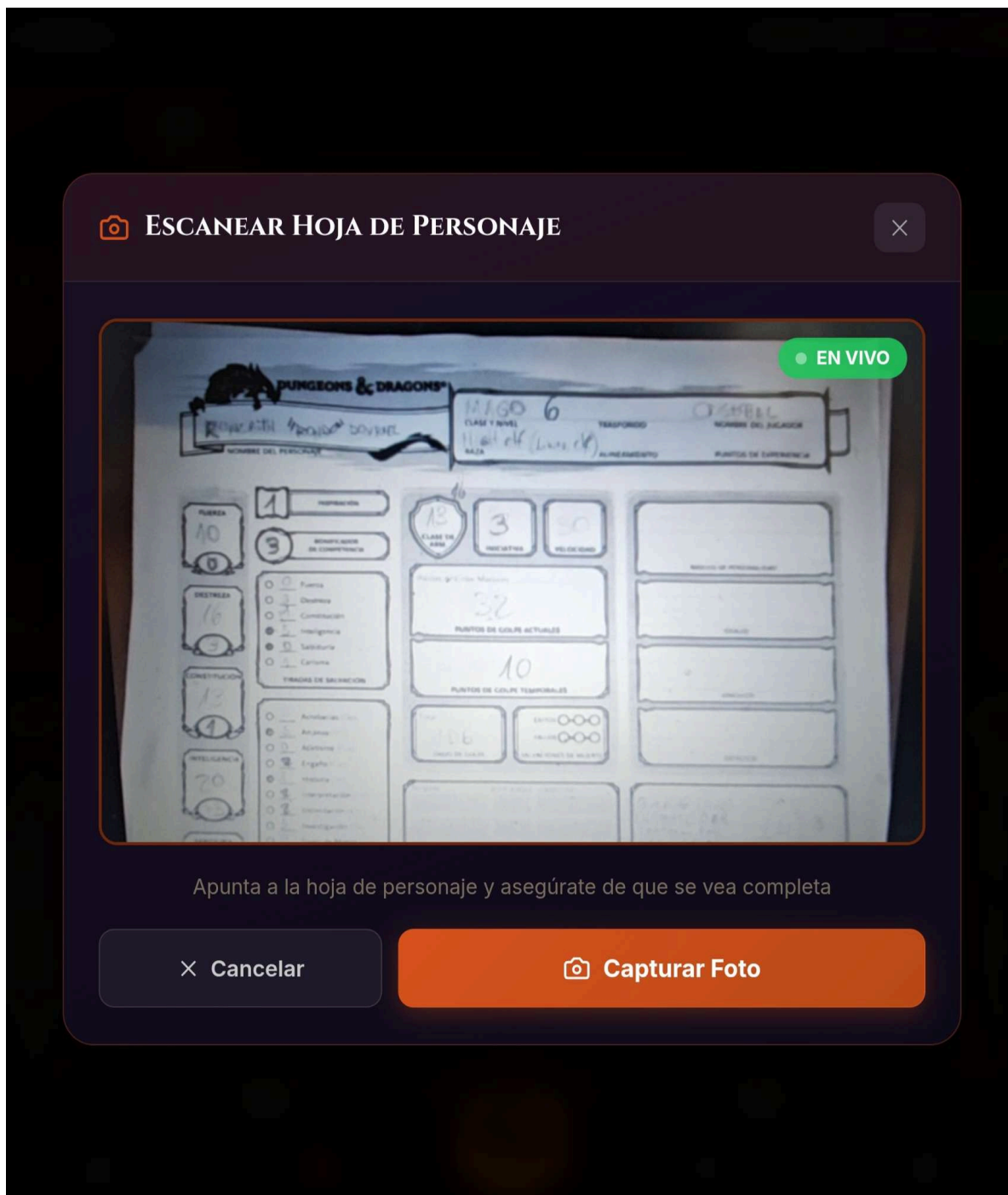
PROFICIENCY BONUS

1

HIT DICE (D6)

Digitalizador OCR de Hojas de Personaje

- Vista de cámara full-screen con máscara semi-transparente para encuadrar la hoja física desde **mobile**.



Modal de revisión (OCRReviewModal.jsx)

- Cuenta con campos de formulario coloreados según la confianza del reconocimiento:
 - **Verde**: alta confianza — atributo validado y listo (ej. Fuerza: 18).
 - **Amarillo**: confianza media — requiere confirmación del usuario (ej. Dote: 'Alert').
 - **Rojo**: confianza baja o campo ausente — requiere entrada manual (ej. Trasfondo).



REVISAR OCR

Editar y confirmar — los campos coinciden con la hoja de personaje exactamente

☒ Alta confianza — 85%

X CERRAR

CHARACTER IDENTITY

CHARACTER NAME

LEVEL

RACE

CLASS

SUBCLASS

BACKGROUND

ALIGNMENT

PLAYER NAME

EXPERIENCE

ABILITY SCORES

STR

DEX

CON

INT

WIS

CHA

PROF BONUS

COMBAT

ARMOR CLASS

INITIATIVE

SPEED

HIT POINTS

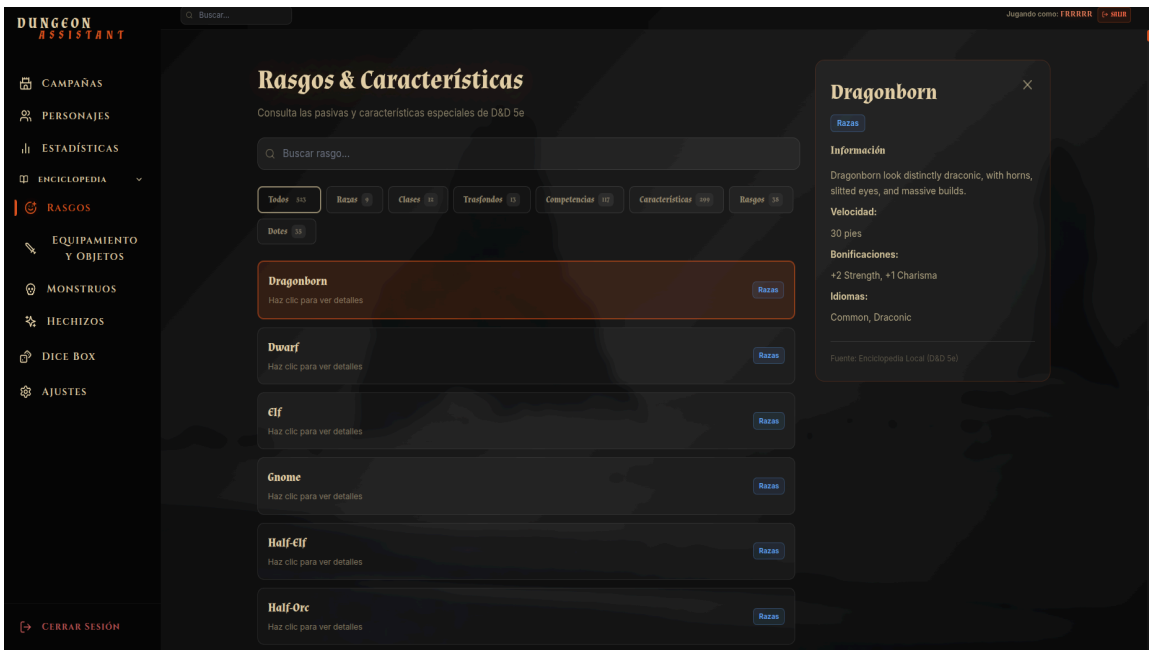
TEMP HP

HIT DICE

REANALIZAR

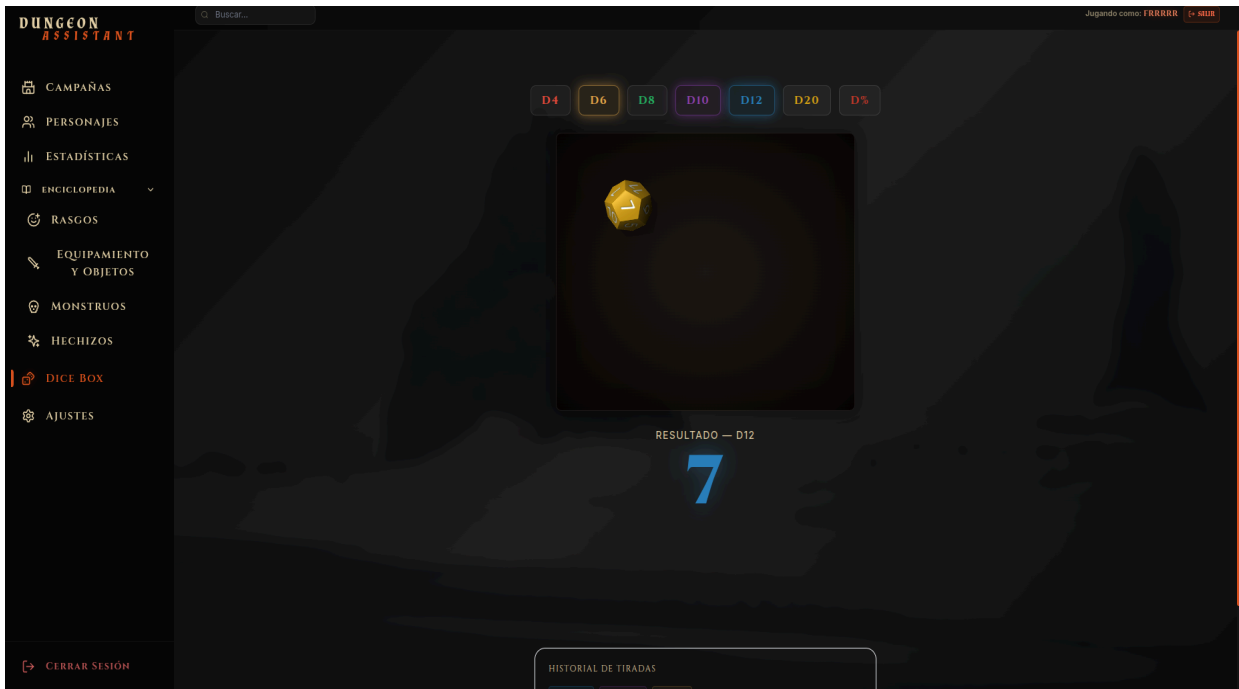
→ ACTUALIZAR HOJA DE PERSONAJE

Enciclopedia interactiva



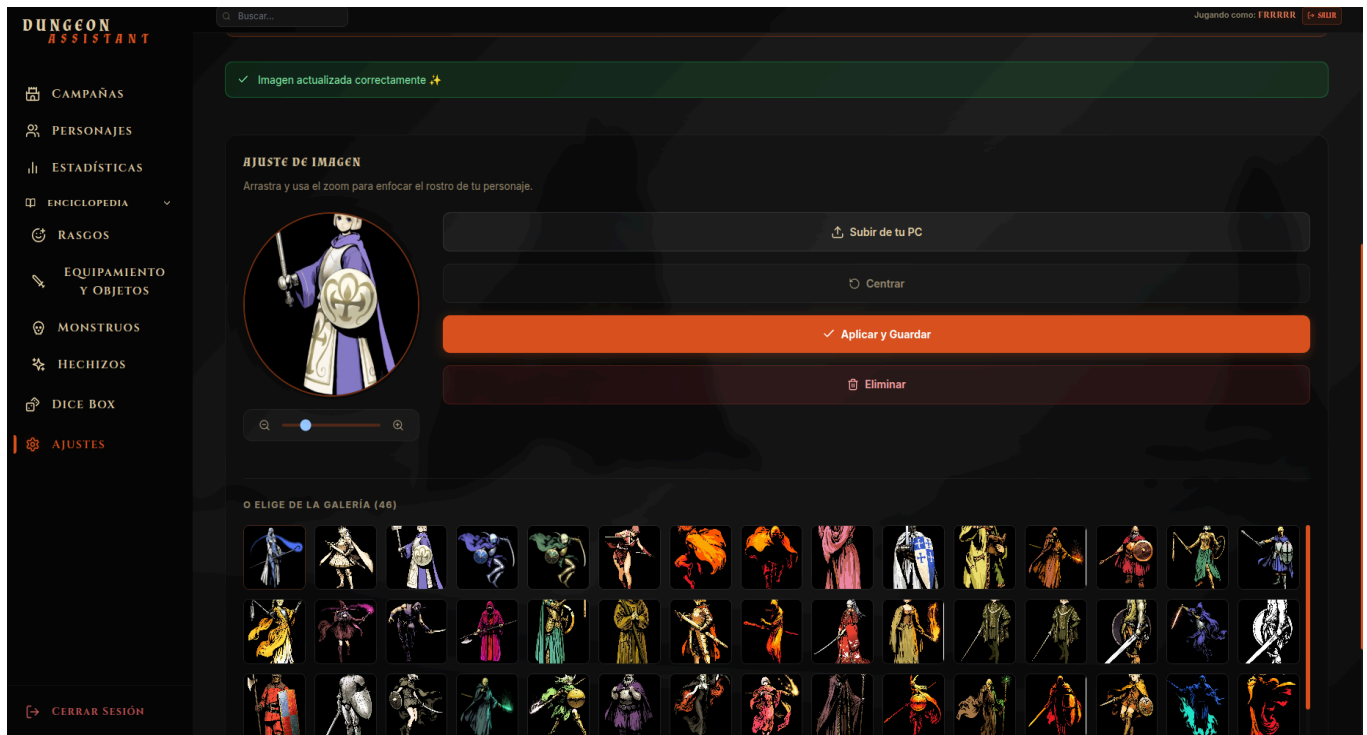
Dados virtuales con diversas caras

- Dado que complementa la mecánica de lanzar dados



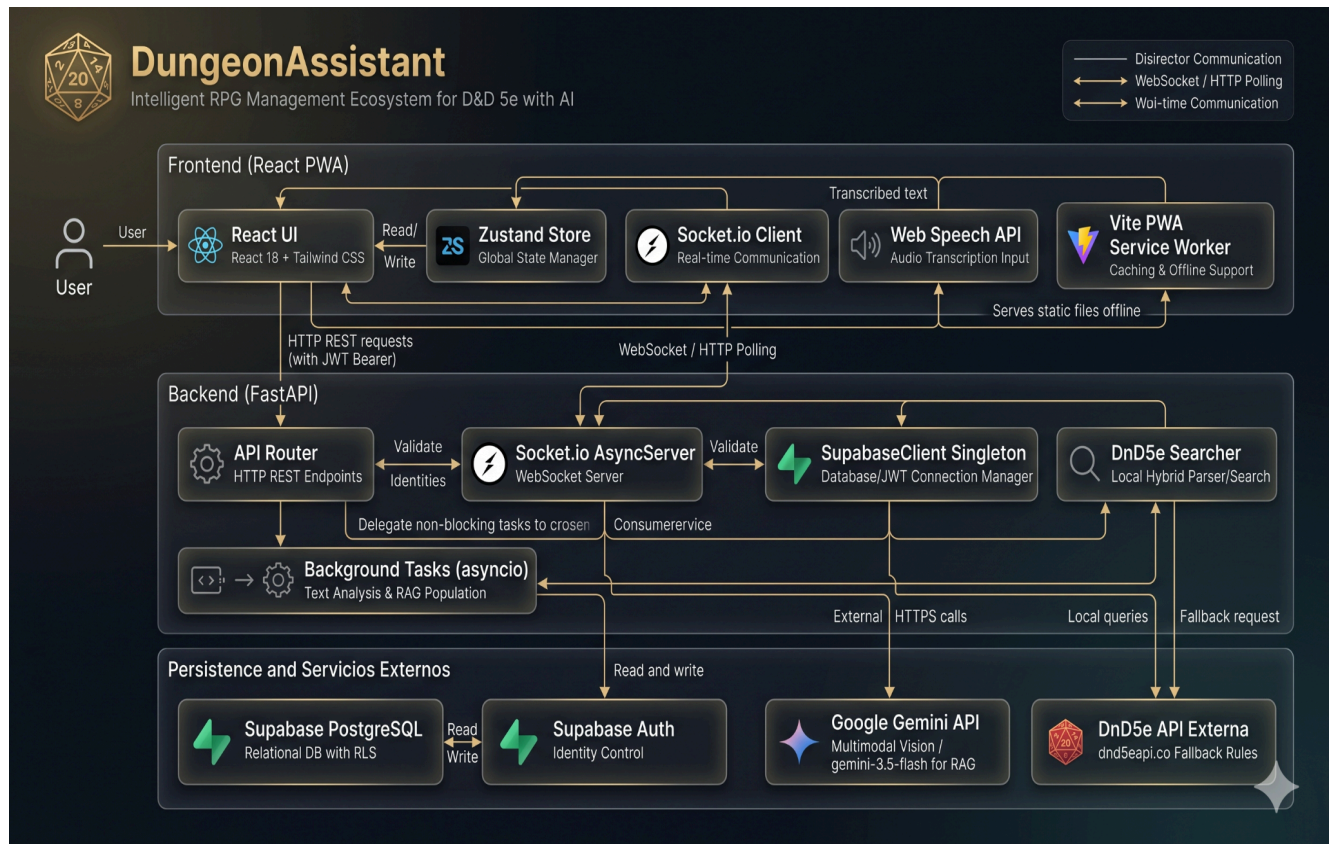
Menú de ajustes:

- Para la personalización de perfil de personaje



3.8 Anexos

Diagrama de Arquitectura General



4. Plan de pruebas

El objetivo principal de nuestras pruebas fueron validar funcionalidad, rendimiento y seguridad de nuestra aplicación antes del lanzamiento.

Sobre el alcance del plan de pruebas de nuestro proyecto:

- Backend: 7 Módulos
- Frontend: 9 Módulos
- Integraciones críticas: Supabase, Google Gemini API, dnd5eapi.com, WebSockets.

4.1 Alcance de pruebas

- Autenticación y autorización de usuarios
- Gestión de campañas y asignación de roles
- Hojas de personaje interactivas
- Generación de NPCs mediante IA (Gemini API)
- OCR para digitalización de hojas físicas
- Asistente conversacional con contexto
- Sincronización en tiempo real (WebSockets)
- Funcionales PWA y modo offline
- Visor de datos 3D interactivo

4.2 Pruebas fuera de alcance

- Pruebas de carga (>100 usuarios en simultáneo)
- Penetration testing avanzado
- Compatibilidades con navegadores obsoletos
- Dispositivos móviles con sistema operativo >2 años de antigüedad

4.3 Estrategia de Pruebas y Metodología

Tipo de Prueba	Herramientas	Alcance
Unitarias	Pytest (Backend), Jest (Frontend)	Validación de funciones individuales con una cobertura objetivo superior al 85%.
Integración	Pytest + Fixtures	Verificación de la comunicación e interacción entre módulos.
API	Pytest, HTTPX, Postman	Validación de endpoints REST, códigos de estado y respuestas HTTP.
WebSocket	socket.io-client	Pruebas de comunicación en tiempo real y mecanismos de reconexión.
End-to-End (E2E)	Playwright	Validación de flujos completos de usuario en múltiples navegadores.
UI/UX	Evaluación manual + Lighthouse	Revisión de responsividad, accesibilidad y contraste visual.
Seguridad	Evaluación manual + OWASP ZAP	Análisis de vulnerabilidades OWASP Top 10, inyecciones, JWT y configuración CORS.

4.4 Priorización de Componentes

Prioridad	Componentes	Impacto
P1 – Crítica	Login y registro, creación de campañas, estabilidad WebSocket, carga de personajes, endpoint de salud (<i>health check</i>)	Funcionalidades indispensables para la operación del sistema.
P2 – Alta	Gestión de roles, edición de personajes, generación de NPC, OCR y sincronización en tiempo real	Impactan directamente la experiencia del usuario.
P3 – Media	Elementos visuales, datos 3D, sistema de crónicas, búsqueda D&D 5e, funcionamiento PWA offline	Funcionalidades complementarias que aportan valor, pero no son críticas.
P4 – Baja	Animaciones visuales, mensajes de error secundarios y casos límite menores	Mejoras orientadas al refinamiento y pulido visual.

4.5 Resumen Ejecutivo

El presente Plan de Pruebas tiene como objetivo asegurar la calidad, estabilidad, seguridad y correcto funcionamiento de Dungeon Assistant, una Progressive Web App para la gestión de campañas de Dungeons & Dragons 5e con integración de inteligencia artificial. La estrategia se basa en un enfoque de pruebas orientado a riesgos, priorizando los componentes críticos del sistema, tales como autenticación, gestión de campañas, sincronización en tiempo real mediante WebSockets y administración de personajes.

El alcance contempla pruebas funcionales, de integración, API, experiencia de usuario, seguridad y validación end-to-end sobre la totalidad de los módulos backend y frontend, así como sus integraciones externas (Supabase, Gemini API y dnd5eapi.co). Se ejecutarán casos de prueba automatizados y manuales, con criterios de aceptación orientados a garantizar una cobertura superior al 95% de los escenarios críticos y una cobertura de código mínima del 85% en los componentes principales.

La ejecución se desarrollará durante un período de 20 días, incorporando actividades de planificación, diseño, ejecución, verificación y cierre. El éxito del proceso se medirá mediante indicadores de calidad definidos, asegurando que el producto alcance los estándares requeridos para su liberación, minimizando riesgos operacionales y garantizando una experiencia confiable para los usuarios finales.

5 Conclusión del proyecto

El desarrollo de **Dungeon Assistant** representó mucho más que la implementación de una aplicación web; constituyó un proceso integral de aprendizaje donde se aplicaron conocimientos de ingeniería de software, arquitectura de sistemas, desarrollo web moderno e integración de servicios de inteligencia artificial para resolver un problema real dentro del ámbito de los juegos de rol de mesa. A lo largo del proyecto fue necesario comprender no sólo cómo desarrollar funcionalidades individuales, sino también cómo hacer que múltiples tecnologías trabajaran de forma coordinada para ofrecer una experiencia consistente, escalable y orientada al usuario.

Uno de los aspectos más relevantes fue la adopción de la **metodología ágil Kanban**, la cual permitió organizar el trabajo mediante un flujo continuo de tareas, facilitando la priorización de funcionalidades, el seguimiento del progreso y la adaptación a cambios durante el desarrollo. Esta metodología resultó especialmente adecuada para un equipo pequeño, ya que proporcionó visibilidad permanente del estado del proyecto, fomentó la colaboración entre los integrantes y permitió distribuir las responsabilidades según las fortalezas técnicas de cada uno, manteniendo un ritmo constante de avance.

Asimismo, el proyecto permitió profundizar en conceptos fundamentales de la ingeniería de software, como el diseño de arquitecturas desacopladas, la aplicación de patrones de diseño (Singleton, Dependency Injection, Service Layer, Middleware y State Management), el desarrollo asíncrono, la seguridad basada en autenticación JWT y políticas **Row Level Security (RLS)**, además del diseño de APIs REST y comunicación en tiempo real mediante **WebSockets**. Estos conceptos fueron aplicados en un contexto práctico, comprendiendo no solo su funcionamiento teórico, sino también las ventajas que aportan en términos de mantenibilidad, rendimiento, escalabilidad y facilidad de pruebas.

La elección de un **stack tecnológico diverso** respondió a las necesidades específicas del problema y permitió comprender que cada tecnología cumple un propósito dentro de una arquitectura moderna. **React** proporcionó una interfaz dinámica y reutilizable; **FastAPI** permitió construir un backend asíncrono de alto rendimiento; **Supabase** ofreció una solución integral para autenticación, base de datos y almacenamiento; **Socket.io** resolvió la necesidad de sincronización en tiempo real entre jugadores; las capacidades de **Google Gemini** hicieron posible incorporar funciones de inteligencia artificial y OCR; mientras que tecnologías como **Web Speech API** y el modelo **PWA** mejoraron la accesibilidad, movilidad y experiencia del usuario. La integración de todas estas herramientas demostró que un sistema moderno requiere seleccionar tecnologías complementarias, donde cada una aporta capacidades específicas que, en conjunto, permiten construir una solución robusta y funcional.

Otro aprendizaje significativo fue comprender la importancia de la **integración entre sistemas externos**, enfrentando desafíos relacionados con autenticación, consumo de APIs, tolerancia a fallos, límites de servicios de IA, despliegue en la nube y sincronización de información entre frontend, backend y servicios externos. Esto permitió adquirir experiencia en problemas propios del desarrollo profesional que van más allá de la programación de funcionalidades aisladas.

Finalmente, el proyecto evidenció que el desarrollo de software de calidad no depende únicamente de escribir código, sino también de aplicar una metodología adecuada, diseñar una arquitectura sólida, seleccionar correctamente las tecnologías, implementar mecanismos de seguridad, planificar pruebas y mantener una visión centrada en el usuario. En conjunto, Dungeon Assistant representa el resultado de la integración de conocimientos adquiridos durante la formación, demostrando la capacidad del equipo para diseñar, desarrollar e implementar una solución tecnológica moderna que combina desarrollo web, computación en la nube, inteligencia artificial y comunicación en tiempo real, sentando además una base sólida para futuras mejoras y la evolución del proyecto.